

1. Աշխատանքի նպատակ

- **Կառուցվածքների (struct)** և դրանց ներդրված տեսակների ստեղծման ու կիրառման հմտությունների ամրապնդում:
- **Կառուցվածքների զանգվածների** հետ աշխատանքի, տվյալների որոնման և դասակարգման (սորտավորման) ալգորիթմների իրականացում:
- **Ֆունկցիաներին պարամետրերի փոխանցման եղանակների** (ըստ հղման &, ըստ const հղման const &, ըստ ցուցիչի *) արդյունավետ և ճիշտ կիրառում:
- Մուտքային տվյալների վավերականության ստուգման (data validation) պրակտիկայի տիրապետում:

2. Տեսական մաս

- **Կառուցվածք (Struct).** Օբյեկտին ուղղված ծրագրավորման մեջ կառուցվածքը օգտագործողի կողմից սահմանվող տվյալների տիպ է, որը թույլ է տալիս մեկ ընդհանուր անվան տակ խմբավորել տարբեր տիպի փոփոխականներ (դաշտեր):
- **Ներդրված կառուցվածքներ (Nested Structs).** Կառուցվածքի դաշտերից մեկը կարող է ինքնին լինել այլ կառուցվածքային տիպ (օրինակ՝ Flight-ի մեջ Time տիպի դաշտ):
- **Փոխանցման եղանակներ.**
 - **const T& (Const reference)** — Օգտագործվում է միայն ընթերցող ֆունկցիաներում: Այն կանխում է տվյալների ավելորդ պատճենումը հիշողության մեջ և երաշխավորում է, որ ֆունկցիան չի փոխի օբյեկտի վիճակը:
 - **T& կամ T* (Reference / Pointer)** — Օգտագործվում է տվյալները փոփոխող ֆունկցիաներում (օրինակ՝ տեղերի ամրագրում կամ դասակարգում):
- **Կառուցվածքային տարրերի փոխանակում.** Դասակարգման (սորտավորման) ընթացքում ամբողջական Flight օբյեկտները փոխանակվում են std::swap-ի կամ ժամանակավոր փոփոխականի միջոցով:

3. Խնդրի պահանջ

1. Սահմանել Time (ժամ, րոպե) և Flight (համար, ուղղություն, մեկնելու ժամ, ազատ տեղեր) կառուցվածքները:

2. Կազմակերպել տվյալների մուտքագրում՝ ստուգելով թվային սահմանները (ժամեր՝ 0-23, րոպեներ՝ 0-59, ազատ տեղեր ≥ 0):
3. Իրականացնել ֆունկցիա, որը որոնում է տրված ուղղության թռիչքները:
4. Իրականացնել ֆունկցիա, որը դասակարգում (սորտավորում) է թռիչքները ըստ մեկնման ժամի:
5. Իրականացնել ֆունկցիա, որը հղումով փոխանցված Flight օբյեկտի համար ամրագրում է տեղ (եթե ազատ տեղեր առկա են):
6. Արտածել մշակված արդյունքները ընթեռնելի աղյուսակի/վերնագրերի տեսքով:

4. C++ Ծրագրի կոդը

C++

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
#include <iomanip>
```

```
#include <algorithm>
```

```
// 1. Ներդրված Time կառուցվածք
```

```
struct Time {
```

```
    int hour; // 0 - 23
```

```
    int minute; // 0 - 59
```

```
    // Օժանդակ ֆունկցիա՝ ժամերը րոպեների վերածելու համար (համեմատման համար)
```

```
    int toMinutes() const {
```

```
        return hour * 60 + minute;
```

```
    }
```

```
};
```

```
// 1. Flight կառուցվածք՝ իմաստալից անգլերեն անուններով
```

```
struct Flight {
```

```

std::string flightNumber;

std::string destination;

Time departureTime;

int availableSeats;

};

// --- ՄՏՈՒԳՈՂ ԵՎ ՄՈՒՏՔԱԳՐՈՂ ՖՈՒՆԿՑԻԱՆԵՐ ---

// Ստուգում է ժամանակի վավերականությունը
bool isValidTime(int h, int m) {
    return (h >= 0 && h < 24) && (m >= 0 && m < 60);
}

// Մուտքագրում է մեկ թռիչքի տվյալները՝ ստուգումներով
void inputFlight(Flight& flight) {
    std::cout << "Flight Number: ";
    std::cin >> flight.flightNumber;
    std::cout << "Destination: ";
    std::cin.ignore();
    std::getline(std::cin, flight.destination);
    // Ժամի մուտքագրում և ստուգում (Պահանջ 4)
    do {
        std::cout << "Departure Time (Hour [0-23] and Minute [0-59]): ";
        std::cin >> flight.departureTime.hour >> flight.departureTime.minute;
        if (!isValidTime(flight.departureTime.hour, flight.departureTime.minute)) {
            std::cout << "Invalid time! Please re-enter.\n";
        }
    } while (!isValidTime(flight.departureTime.hour, flight.departureTime.minute));

    // Ազատ տեղերի մուտքագրում և ստուգում (Պահանջ 4)
    do {

```

```

std::cout << "Available Seats (>= 0): ";

std::cin >> flight.availableSeats;

if (flight.availableSeats < 0) {

    std::cout << "Seats count cannot be negative! Please re-enter.\n";

}

} while (flight.availableSeats < 0);

}

// --- ԸՆԹԵՐՑՈՂ ՖՈՒՆԿՑԻԱՆԵՐ (const reference) ---

// Արտածում է թռիչքների ցուցակը ձևավորված տեսքով (Պահանջ 3, 6)

void printFlights(const std::vector<Flight>& flights) {

    if (flights.empty()) {

        std::cout << "No flights available.\n";

        return;

    }

    std::cout << "\n-----\n";

    std::cout << std::left << std::setw(15) << "Flight No."

        << std::setw(20) << "Destination"

        << std::setw(15) << "Dep. Time"

        << std::setw(10) << "Seats" << "\n";

    std::cout << "-----\n";

    for (const auto& flight : flights) {

        std::cout << std::left << std::setw(15) << flight.flightNumber

            << std::setw(20) << flight.destination

            << std::right << std::setfill('0') << std::setw(2) << flight.departureTime.hour << ":"

            << std::setw(2) << flight.departureTime.minute << std::setfill(' ')

            << std::setw(10) << ""

            << std::setw(5) << flight.availableSeats << "\n";

    }

```

```
std::cout << "-----\n";  
}
```

// Որոնում է թռիչքները ըստ ուղղության (Պահանջ 2, 3)

```
void searchByDestination(const std::vector<Flight>& flights, const std::string& dest) {  
    std::cout << "\nSearch Results for Destination: \"" << dest << "\"\n";  
    std::vector<Flight> results;  
    for (const auto& flight : flights) {  
        if (flight.destination == dest) {  
            results.push_back(flight);  
        }  
    }  
    printFlights(results);  
}
```

// --- ՓՈՓՈԽՈՂ ՖՈԻՆԿՑԻԱՆԵՐ (reference) ---

// Դասակարգում (սորտավորում) է թռիչքները ըստ ժամի (Պահանջ 2, 3, 5)

```
void sortByDepartureTime(std::vector<Flight>& flights) {  
    int n = flights.size();  
    for (int i = 0; i < n - 1; ++i) {  
        for (int j = 0; j < n - i - 1; ++j) {  
            if (flights[j].departureTime.toMinutes() > flights[j + 1].departureTime.toMinutes()) {  
                // Փոխանակում ենք ամբողջ կառուցվածքային տարրը (Պահանջ 5)  
                std::swap(flights[j], flights[j + 1]);  
            }  
        }  
    }  
}
```

```

    std::cout << "\n[INFO] Flights successfully sorted by departure time.\n";
}

// Ամրագրում է տեղ տրված թռիչքի համար հղումով (Պահանջ 3)
bool bookSeat(Flight& flight) {
    if (flight.availableSeats > 0) {
        flight.availableSeats--;

        std::cout << "\n[SUCCESS] Seat successfully booked for flight " <<
flight.flightNumber << "!\n";

        return true;
    } else {
        std::cout << "\n[ERROR] Booking failed! No available seats on flight " <<
flight.flightNumber << ".\n";

        return false;
    }
}

// --- MAIN ՖՈՒՆԿՑԻԱ ---
int main() {
    int count = 0;

    // Քանակի ստուգում (Պահանջ 4)
    std::cout << "Enter the number of flights: ";
    std::cin >> count;

    while (count <= 0) {
        std::cout << "Count must be positive. Enter again: ";
        std::cin >> count;
    }

    std::vector<Flight> schedule(count);

    // Տվյալների մուտքագրում
    for (int i = 0; i < count; ++i) {

```

```

        std::cout << "\n--- Entering data for flight #" << (i + 1) << " ---\n";

        inputFlight(schedule[i]);
    }

    // 1. Արտածել նախնական ցուցակը
    std::cout << "\n=== INITIAL FLIGHT SCHEDULE ===";
    printFlights(schedule);

    // 2. Դասակարգում ըստ մեկնելու ժամի
    sortByDepartureTime(schedule);
    std::cout << "\n=== SORTED FLIGHT SCHEDULE ===";
    printFlights(schedule);

    // 3. Որոնում ըստ ուղղության
    std::string searchDest;
    std::cout << "\nEnter destination to search for: ";
    std::cin.ignore();
    std::getline(std::cin, searchDest);
    searchByDestination(schedule, searchDest);

    // 4. Ամրագրման թեստավորում (հղումով փոխանցման ցուցադրություն)
    if (!schedule.empty()) {
        std::cout << "\nTesting booking system for flight " << schedule[0].flightNumber <<
        "...";

        bookSeat(schedule[0]); // Ամրագրում ենք առաջին թռիչքի համա
        std::cout << "\n=== SCHEDULE AFTER BOOKING ===";
        printFlights(schedule);
    }

    return 0;
}

```